

```

import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
import { getAuth, onAuthStateChanged } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";
import { getDatabase, ref, set, onValue, remove, get, child, update } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-database.js";

const firebaseConfig = {
  apiKey: "AIzaSyB0gShB0u05UszCBLS-bpTl2f3AI7_I-pY",
  authDomain: "reservasisd.firebaseio.com",
  databaseURL: "https://reservasisd-default-rtdb.firebaseio.com",
  projectId: "reservasisd",
  storageBucket: "reservasisd.firebaseio.com",
  messagingSenderId: "637702189208",
  appId: "1:637702189208:web:49ff477b35e299564ca0ed"
};

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db = getDatabase(app);

// ELEMENTOS DEL DOM
const formNuevo = document.getElementById('form-nuevo-recurso');
const listaInventario = document.getElementById('lista-inventario');
const btnVolver = document.getElementById('btn-volver');

// NUEVOS ELEMENTOS DEL DOM PARA ADMINS
const formNuevoAdmin = document.getElementById('form-nuevo-admin');
const listaAdministradores = document.getElementById('lista-administradores');

// NUEVOS ELEMENTOS DEL DOM PARA PROFESORES
const formNuevoProfesor = document.getElementById('form-nuevo-profesor');
const listaProfesores = document.getElementById('lista-profesores');

// 1. GUARDIAN Estricto Dinámico
onAuthStateChanged(auth, async (user) => {
  if (!user) {
    window.location.href = "index.html";
    return;
  }

  const email = user.email;

  // EXCEPCIÓN DIRECTA: Si sos vos, entrás de una
  if (email === "laggerro2@gmail.com") {
    inicializarPanel();
    return;
  }

  // Para cualquier otro correo, verificamos en la RTDB

```

```

try {
  const emailLimpio = email.replace(/\.\/g, '_');
  const dbRef = ref(db);
  const snapshotAdmin = await get(child(dbRef,
`administradores/${emailLimpio}`));

  if (snapshotAdmin.exists()) {
    inicializarPanel();
  } else {
    alert("Acceso denegado. Se requieren credenciales de administrador.");
    window.location.href = "equipos.html";
  }
} catch (error) {
  console.error("Error al verificar permisos de administrador:", error);
  window.location.href = "equipos.html";
}
});

// Modificamos la función inicializadora para cargar todo
function inicializarPanel() {
  escucharYListarEquipos();
  escucharYListarAdmins();
  escucharYListarProfesores();
}

// =====
// MÓDULO A: CONTROL DE INVENTARIO (EQUIPOS)
// =====

// ENVIAR FORMULARIO (SUBIDA PHP + REGISTRO FIREBASE)
formNuevo.addEventListener('submit', async (e) => {
  e.preventDefault();
  const categoria = document.getElementById('recurso-categoria').value;
  const nombre = document.getElementById('recurso-nombre').value.trim();
  const cantidad = parseInt(document.getElementById('recurso-cantidad').value);
  const descripcion = document.getElementById('recurso-descripcion').value.trim();
  const inputImagen = document.getElementById('recurso-imagen').files[0];
  const idRecurso = nombre.toLowerCase()
    .replace(/^[a-z0-9]+/g, '_')
    .replace(/(^|_|$)/g, '');

  try {
    const formData = new FormData();
    formData.append('imagen', inputImagen);

    const responsePhp = await fetch('subir-imagen.php', {
      method: 'POST',
      body: formData
    });
  }
});

```

```

const dataPhp = await responsePhp.json();

if (dataPhp.status !== 'success') {
  throw new Error(dataPhp.message || "Error al subir la imagen al servidor
XAMPP.");
}

const rutaImagenServidor = dataPhp.ruta;
const recursoRef = ref(db, `inventario/${categoria}/${idRecurso}`);

await set(recursoRef, {
  nombre: nombre,
  stock_total: cantidad,
  descripcion: descripcion,
  imagen: rutaImagenServidor
});

alert("¡Equipo agregado con éxito al inventario!");
formNuevo.reset();
} catch (error) {
  console.error("Error al registrar:", error);
  alert(`Falló el registro: ${error.message}`);
}
});

// LISTAR EQUIPOS
function escucharYListarEquipos() {
  const inventarioRef = ref(db, 'inventario');
  onValue(inventarioRef, (snapshot) => {
    listaInventario.innerHTML = '';
    const categorias = snapshot.val();
    if (!categorias) {
      listaInventario.innerHTML = '<p class="text-center text-gray-400 py-4">No hay
equipos en el inventario.</p>';
      return;
    }

    Object.keys(categorias).forEach(idCategoria => {
      const recursos = categorias[idCategoria];
      Object.keys(recursos).forEach(idRecurso => {
        const item = recursos[idRecurso];
        const itemHTML = `
      <div class="flex items-center justify-between p-3 bg-gray-50 border
rounded-xl hover:shadow-sm transition-all">
        <div class="flex items-center gap-3">
          
          <div>
            <h4 class="font-bold text-gray-800 text-sm
leading-tight">${item.nombre}</h4>

```

```

        <p class="text-xs text-gray-500 uppercase tracking-wide
font-semibold">${idCategoria} - Stock: ${item.stock_total}</p>
    </div>
</div>
    <button onclick="eliminarRecurso('${idCategoria}', '${idRecurso}')"
class="text-red-500 hover:text-red-700 p-2 transition">
        <i class="fa-solid fa-trash-can"></i>
    </button>
</div>
`;
    listaInventario.insertAdjacentHTML('beforeend', itemHTML);
});
});
});
}

// ELIMINAR RECURSO
window.eliminarRecurso = async function(categoria, idRecurso) {
    const confirmar = confirm("¿Estás seguro de que quieres eliminar este recurso del
inventario de forma permanente? Se perderán sus datos de stock.");
    if (confirmar) {
        try {
            const recursoRef = ref(db, `inventario/${categoria}/${idRecurso}`);
            await remove(recursoRef);
            alert("Recurso eliminado del inventario con éxito.");
        } catch (error) {
            console.error("Error al eliminar:", error);
            alert("No se pudo eliminar el recurso.");
        }
    }
};

// =====
// MÓDULO B: CONTROL DE ADMINISTRADORES (NUEVO)
// =====

// AGREGAR NUEVO ADMINISTRADOR
formNuevoAdmin.addEventListener('submit', async (e) => {
    e.preventDefault();
    const nombre = document.getElementById('admin-nombre').value.trim();
    const email = document.getElementById('admin-email').value.trim().toLowerCase();
    const emailLimpio = email.replace(/\.\/g, '_');

    try {
        const adminRef = ref(db, `administradores/${emailLimpio}`);
        await set(adminRef, {
            nombre: nombre,
            email: email
        });
        alert(`Se han asignado privilegios de administrador a ${nombre} de manera

```

```

exitosa.`);
    formNuevoAdmin.reset();
  } catch (error) {
    console.error("Error al agregar administrador:", error);
    alert("Ocurrió un error al registrar el administrador en la base de datos.");
  }
});

// ESCUCHAR Y LISTAR ADMINISTRADORES
function escucharYListarAdmins() {
  const adminsRef = ref(db, 'administradores');
  onValue(adminsRef, (snapshot) => {
    listaAdministradores.innerHTML = '';
    const admins = snapshot.val();
    if (!admins) {
      listaAdministradores.innerHTML = '<p class="text-center text-gray-400 py-4">No hay administradores registrados.</p>';
      return;
    }

    Object.keys(admins).forEach(key => {
      const admin = admins[key];
      const esUsuarioPrincipal = admin.email === "laggerro2@gmail.com";

      const botonBorrar = esUsuarioPrincipal
        ? `<span class="text-xs bg-blue-100 text-blue-800 px-2.5 py-0.5 rounded-full font-semibold uppercase">Propietario</span>`
        : `<button onclick="eliminarAdmin('${key}', '${admin.nombre}')" class="text-red-500 hover:text-red-700 p-2 transition">
          <i class="fa-solid fa-user-minus"></i>
        </button>`;

      const itemHTML = `
        <div class="flex items-center justify-between p-3 bg-gray-50 border rounded-xl hover:shadow-sm transition-all">
          <div class="flex items-center gap-3">
            <div class="w-10 h-10 bg-indigo-100 rounded-full flex items-center justify-center text-indigo-700">
              <i class="fa-solid fa-user-shield"></i>
            </div>
            <div>
              <h4 class="font-bold text-gray-800 text-sm leading-tight">${admin.nombre}</h4>
              <p class="text-xs text-gray-500 font-medium">${admin.email}</p>
            </div>
            <div>
              ${botonBorrar}
            </div>
          </div>
        </div>
      `;
      listaAdministradores.insertAdjacentHTML('beforeend', itemHTML);
    });
  });
}

```

```

    });
  });
}

```

```

// BORRAR ADMINISTRADOR

```

```

window.eliminarAdmin = async function (emailLimpio, nombreAdmin) {
  const confirmar = confirm(`¿Estás seguro de que querés retirarle los permisos de
administrador a "${nombreAdmin}"?\nEsta cuenta ya no podrá acceder a este panel de
ajustes ni ver el botón de configuración.`);
  if (confirmar) {
    try {
      const adminRef = ref(db, `administradores/${emailLimpio}`);
      await remove(adminRef);
      alert(`Permisos revocados para ${nombreAdmin} correctamente.`);
    } catch (error) {
      console.error("Error al revocar permisos:", error);
      alert("No se pudo dar de baja el administrador.");
    }
  }
};

```

```

// BOTÓN VOLVER

```

```

btnVolver.addEventListener('click', () => {
  window.location.href = "equipos.html";
});

```

```

// =====
// MÓDULO C: CONTROL DE PROFESORES (LISTA BLANCA)
// =====

```

```

// REGISTRAR PROFESOR EN LA LISTA BLANCA

```

```

formNuevoProfesor.addEventListener('submit', async (e) => {
  e.preventDefault();
  const nombre = document.getElementById('profe-nombre').value.trim();
  const email = document.getElementById('profe-email').value.trim().toLowerCase();
  const emailLimpio = email.replace(/\./g, '_');

  try {
    const profeRef = ref(db, `usuarios_autorizados/${emailLimpio}`);
    await set(profeRef, {
      nombre: nombre,
      email: email,
      recibe_reporte: false // Por defecto se crea sin recibir reportes
    });
    alert(`El docente ${nombre} ha sido agregado a la lista autorizada.`);
    formNuevoProfesor.reset();
  } catch (error) {
    console.error("Error al registrar profesor:", error);
    alert("Ocurrió un error al guardar el docente.");
  }
});

```

```

    }
  });

// ESCUCHAR Y LISTAR PROFESORES
function escucharYListarProfesores() {
  const profesoresRef = ref(db, 'usuarios_autorizados');
  onValue(profesoresRef, (snapshot) => {
    listaProfesores.innerHTML = '';
    const profesores = snapshot.val();
    if (!profesores) {
      listaProfesores.innerHTML = '<p class="text-center text-gray-400 py-4">No hay docentes autorizados en la lista.</p>';
      return; // El return correcto va dentro del "if" únicamente
    }

    Object.keys(profesores).forEach(key => {
      const profe = profesores[key];
      const esPropietario = profe.email === "laggerro2@gmail.com";

      const botonBorrar = esPropietario
        ? `<span class="text-xs bg-blue-100 text-blue-800 px-2.5 py-0.5 rounded-full font-semibold">Dueño</span>`
        : `<button onclick="eliminarProfesor('${key}', '${profe.nombre}')" class="text-red-500 hover:text-red-700 p-2 transition">
          <i class="fa-solid fa-user-slash"></i>
        </button>`;

      // Creamos el diseño de la tarjeta integrando el switch/checkbox
      const itemHTML = `
        <div class="flex items-center justify-between p-3 bg-gray-50 border rounded-xl hover:shadow-sm transition-all mb-2">
          <div class="flex items-center gap-3">
            <div class="w-10 h-10 bg-green-100 rounded-full flex items-center justify-center text-green-700">
              <i class="fa-solid fa-chalkboard-user"></i>
            </div>
            <div>
              <h4 class="font-bold text-gray-800 text-sm leading-tight">${profe.nombre}</h4>
              <p class="text-xs text-gray-500 font-medium">${profe.email}</p>

              <!-- Checkbox para activar/desactivar el reporte por correo -->
              <label class="inline-flex items-center gap-1.5 mt-1 cursor-pointer select-none">
                <input type="checkbox" id="check-rep-${key}" class="w-3.5 h-3.5 rounded text-green-600 focus:ring-green-500 cursor-pointer" ${profe.recibe_reporte ? 'checked' : ''}>
                <span class="text-[10px] text-gray-500 font-bold uppercase tracking-wider">¿Recibe Reporte Diario?</span>
              </label>
            </div>
          </div>
        </div>
      `;
    });
  });
}

```

```

        </div>
    </div>
    ${botonBorrar}
</div>
`;

listaProfesores.insertAdjacentHTML('beforeend', itemHTML);

// Añadimos el evento interactivo al checkbox recién creado
const checkReporte = document.getElementById(`check-rep-${key}`);
if (checkReporte) {
    checkReporte.addEventListener('change', async (e) => {
        const nuevoEstado = e.target.checked;
        try {
            const updates = {};
            updates[`usuarios_autorizados/${key}/recibe_reporte`] = nuevoEstado;
            await update(ref(db), updates);
            console.log(`Preferencia de reporte actualizada para ${profe.nombre}:
${nuevoEstado}`);
        } catch (err) {
            console.error("Error al actualizar preferencia de reporte:", err);
            alert("No se pudo actualizar la configuración.");
            e.target.checked = !nuevoEstado; // Revierte el check si hay un error
de conexión
        }
    });
}
});
});
}

// QUITAR PROFESOR DE LA LISTA BLANCA
window.eliminarProfesor = async function (emailLimpio, nombreProfe) {
    const confirmar = confirm(`¿Estás seguro de que querés REVOCAR el acceso a
"${nombreProfe}"?\nSi lo hacés, perderá la capacidad de iniciar sesión de forma
inmediata.`);
    if (confirmar) {
        try {
            // 1. Lo borramos de la lista blanca general
            const profeRef = ref(db, `usuarios_autorizados/${emailLimpio}`);
            await remove(profeRef);

            // 2. Por seguridad, si además era Administrador, también le quitamos el rol
            const adminRef = ref(db, `administradores/${emailLimpio}`);
            await remove(adminRef);

            alert(`Acceso revocado con éxito para ${nombreProfe}.`);
        } catch (error) {
            console.error("Error al revocar acceso:", error);
            alert("No se pudo completar la operación.");
        }
    }
}

```



```
}  
}  
};
```